

Conversation Context in Long Sessions

Long chats don't just get long — they get worse.

IN THIS LECTURE,

1 Why long chats degrade (lost-in-the-middle)

2 Trimming bloated tool output

3 Progressive summarization & its risk

4 The case-facts block

5 Extract & persist

Why Long Chats Degrade: Lost in the Middle

The window fills up — and the middle fades.

The context window is finite — and fills up

- ▶ As it fills, accuracy drops — this is 'context rot'
- ▶ Claude weights the START and END most
- ▶ A key fact buried in the MIDDLE can be missed



? Bury a crucial fact in the middle of a huge chat and it can be overlooked — everything here fights that.

Trim Bloat Tool Output

The biggest space-waster is old tool results.



Keep the conclusion, drop the raw dump

- ▶ Every file read / API call dumps a huge result into the chat
- ▶ Most of it is never needed again
- ▶ Trim or clear old tool outputs once you've used them

💡 A 2,000-line file read or API dump you already summarized is pure bloat — clear it, keep the takeaway.

Progressive Summarization (and Its Risk)

Summarize the old parts to make room — carefully.

Condense older turns and replace them

- ▶ This is what /compact does — summarize, then swap in
- ▶ Risk: a careless summary drops a detail you still needed
- ▶ Tell Claude what MUST survive the summary



A bad summary is silent — Claude keeps going as if nothing's missing. Protect the essentials explicitly.

The Case-Facts Block

Pin the essential facts where Claude always sees them.



Keep a live case-facts block — re-inject it at the END

- ▶ A short block: customer ID, the issue, decisions made
- ▶ Re-inject it near the end of the prompt each turn
- ▶ ...where Claude's attention is highest



This defeats lost-in-the-middle directly — the must-know facts live at the end, not buried in the history.

The Core Principle: Extract & Persist

Keep the meaning, drop the noise.

Extract the key facts, persist them, trim everything else

- ▶ Trimming tool output, summarizing, case-facts — all the same move
- ▶ Applies to every long-running task
- ▶ The backbone of this whole domain



🔗 Lecture 5.4 applies this very same idea to exploring a large codebase — same principle, new context.

Key Takeaways

Long-session context, in three lines.

1

Long chats degrade

Context rot + lost-in-the-middle — quality drops as the window fills.

2

Trim & summarize

Clear old tool output; summarize carefully, protecting the essentials.

3

Extract & persist

Pin a case-facts block at the end; keep key facts, trim the rest.



End of Lecture 5.1 — Conversation Context in Long Sessions

Escalation & Ambiguity Resolution

Knowing when to stop and ask — or hand off.

IN THIS LECTURE,

1 Why escalate

2 Escalation triggers (incl. policy gaps)

3 Honoring an explicit human request

4 Clarifying multiple matches

Why Escalate?

A confident wrong action is worse than asking.



Press ahead

- ✗ Guesses when unsure
- ✗ Causes the expensive mistakes

VS



Escalate

- ✓ Hands the decision to a human
- ✓ Correct behaviour, not a failure



The dangerous agent is the one that plows ahead when unsure. Knowing its limits is a feature.

Escalation Triggers

The signals that mean 'stop and get a human.'

Low confidence

Claude isn't sure enough to act



High stakes

Money, safety, anything legal



Policy gap

No rule you gave it covers this case



Policy-gap escalation is exam-tested: when no rule applies, Claude must escalate — not invent a policy.

Honor an Explicit Human Request

If the user asks for a human, give them a human.

Escalate immediately — don't try to resolve it first

- ▶ 'I want to speak to a person' is a valid escalation trigger
- ▶ Don't talk them out of it or try one more time
- ▶ An explicit request outranks your urge to help



Overriding a clear request to talk to a human — however well-meaning — erodes trust fast.

Clarify Multiple Matches

When the request fits more than one thing, ask.



Ask a clarifying question — don't silently pick one

- ▶ Two customers named 'Sharma'? Two orders that fit?
- ▶ Ask: 'Did you mean order #A or #B?'
- ▶ This asks the USER to disambiguate — not a human handoff



Guessing between matches risks acting on the wrong record. One clarifying question prevents a costly error.

Key Takeaways

Escalation & ambiguity, in three lines.

1

Escalate on the signals

Low confidence, high stakes, or a policy gap — never invent a policy.

2

Honor 'I want a human'

An explicit request for a person is an instant escalation.

3

Ambiguous → ask

Multiple matches? Ask a clarifying question instead of guessing.



End of Lecture 5.2 — Escalation & Ambiguity Resolution

Error Propagation in Multi-Agent Systems

When one worker fails, the team shouldn't collapse.

IN THIS LECTURE,

1 The problem

2 The structured error hand-back

3 Failure type, partial results & access-vs-empty

4 Recovery strategies

The Problem: A Subagent Fails

A silent failure is the worst failure.

If it returns nothing, the coordinator can't tell what broke

- ▶ It may build the final answer on missing data
- ▶ ...and report a confident but incomplete result
- ▶ The danger is the silence, not the failure itself



Parts fail — that's normal. The real danger is the coordinator not knowing, and hiding the gap.

The Structured Error Hand-Back

Report the failure up — clearly and in a fixed shape.



Return a structured error to the coordinator

- ▶ What failed, why, and what it DID get — not silence
- ▶ Pass it UP so the coordinator can reason about it
- ▶ Report, don't swallow

↪ Reuse the error format from Lecture 2.2 (isError, category) — here the focus is passing it UP to the coordinator.

Failure Type, Partial Results & Access-vs-Empty

Tell the coordinator exactly what kind of failure this is.



Access failure

- ✗ 'Couldn't reach the source'
- ✗ A real failure — retryable



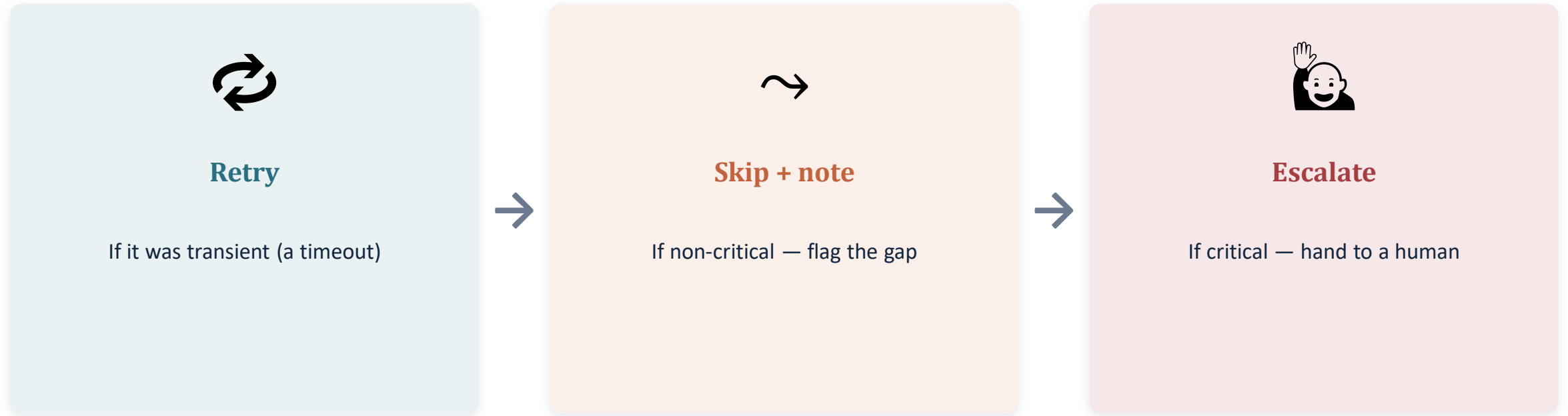
Empty result

- ✓ 'Reached it, found 0 results'
- ✓ A valid answer — do NOT retry

👉 Also carry the failure type + any partial results ('got 2 of 5 sources'). Same trap as 2.2 — empty is an answer, not a failure.

Recovery Strategies

Now the coordinator can actually do something.



💡 Report honestly: 'Findings complete except news sources, which were unavailable' beats a silently-incomplete report.

Key Takeaways

Error propagation, in three lines.

1

Never fail silently

A subagent hands failures UP as a structured error.

2

Name the failure

Failure type + partial results; separate access-failure from empty-result.

3

Recover & be honest

Retry, skip-with-a-note, or escalate — and report what's missing.



End of Lecture 5.3 — Error Propagation in Multi-Agent Systems

Context in Large-Codebase Exploration

Explore a massive codebase without drowning in it.

IN THIS LECTURE,

1 The problem

2 Scratchpad files

3 /compact

4 Crash-recovery manifests

5 The Explore subagent

The Problem: Exploration Floods the Window

Reading a big codebase fills the window fast.

The same context rot as 5.1 — now driven by code

- ▶ Each file read dumps its contents into context
- ▶ Within dozens of files, the window is full
- ▶ Quality drops mid-task



↪ We met the habit-level fix in 2.5 (explore incrementally). This lecture adds the tools for LONG sessions.

Scratchpad Files

Write findings to disk, not just into the chat.



Save key findings to a file

- ▶ 'auth lives in auth/, the bug is in login()'
- ▶ Disk is unlimited; the context window isn't
- ▶ Re-read only the note you need, when you need it



This is extract-and-persist made literal: persist findings to a file, keep context lean.

/compact

Summarize the session to keep going past the limit.

Summarize the conversation and replace the history

- ▶ Shrinks tokens so a long task can continue
- ▶ Progressive summarization from 5.1, with a command
- ▶ Run it proactively at ~70–75% full — you control what it keeps



Compact at a natural break, not only when forced — a clean moment lets you protect what matters.

Crash-Recovery Manifests

If the session dies, don't start over.



Keep a manifest: the task, what's done, what's next

- ▶ A new session reads it and resumes
- ▶ No re-exploring the whole project from scratch
- ▶ Persistence protecting you against failure



A long refactor that crashes at file 40 shouldn't restart at file 1 — the manifest is the safety net.

The Explore Subagent

Send the heavy reading to a separate context.

It reads in its OWN context, returns only a summary

- ▶ Delegate investigation to the Explore subagent (from 3.4)
- ▶ The big file reads never touch your main window
- ▶ Only the conclusions come home



🔗 Taught in 3.4 (plan mode) — here it's a context-management tool: isolation keeps your main window clean.

Putting It Together

Four tools, one lean session.



Explore subagent

Heavy reads happen elsewhere



Scratchpad files

Persist findings to disk



/compact

Shrink the conversation



Manifest

Survive a crash



Same principle as 5.1 — extract, persist, trim — expressed through code-specific tools.

Key Takeaways

Codebase context, in three lines.

1

Exploration floods context

Same context rot as long chats — now driven by code.

2

Persist & compact

Scratchpad files to persist, /compact to summarize, a manifest to survive crashes.

3

Delegate heavy reads

The Explore subagent's separate context keeps your window lean.



End of Lecture 5.4 — Context in Large-Codebase Exploration

Human Review & Confidence Calibration

Trust the sure answers — check the shaky ones.

IN THIS LECTURE,

1 Why human review

2 Field-level confidence

3 Routing by confidence

4 Stratified sampling

5 Accuracy by document type / field

Why Human Review?

You can't check everything — and shouldn't have to.

Review the RIGHT ones — not all, not none

- ▶ Reviewing every output at scale is impossible
- ▶ Reviewing none is risky
- ▶ Target the ones most likely to be wrong



? Smart review isn't 'check everything' or 'trust everything' — it's 'check the ones most likely to be wrong.'

Field-Level Confidence

Not 'is this right?' but 'which parts are shaky?'



Score confidence per field, not per document

- ▶ The typed policy number may be certain...
- ▶ ...while the handwritten total is not
- ▶ Localise the doubt instead of doubting the whole doc



Field-level beats document-level — trust the clear fields, route only the shaky one for review.

Routing by Confidence

Confidence decides the path automatically.

High → auto-accept · low → send to a human

- ▶ The score becomes a routing rule
- ▶ Obvious cases never bother anyone
- ▶ The human's time goes only to uncertain cases



🔗 This is the routing idea from 4.6, now the backbone of the review workflow.

Stratified Sampling

Also spot-check the confident ones — a smart sample.



Sample across every confidence band

- ▶ Don't only review low-confidence items
- ▶ Sample high, medium AND low
- ▶ Catches cases where Claude was confidently wrong



Reviewing only low-confidence items misses confident mistakes — sampling every band keeps confidence itself honest.

Accuracy by Document Type and Field

Learn where Claude is weak — and watch those spots.

Track accuracy by document type and by field

- ▶ Handwritten forms error-prone? The 'total' field weak?
- ▶ Route THOSE for review more aggressively
- ▶ Relax on the fields that are consistently correct



Your review data is a map of weaknesses — feed it back to tighten routing where errors actually cluster.

Key Takeaways

Human review & confidence, in three lines.

1

Review the right ones

Not all, not none — target the outputs most likely to be wrong.

2

Field-level + routing

Score confidence per field; high → auto, low → human.

3

Sample & track

Stratified-sample every band; track accuracy by type/field to aim review.



End of Lecture 5.5 — Human Review & Confidence Calibration

Provenance & Multi-Source Synthesis

Every claim should be able to say where it came from.

IN THIS LECTURE,

1 The problem

2 Claim-source mapping

3 Conflict annotation + uncertainty

4 Temporal framing

The Problem: Facts Lose Their Source

Blend ten sources and attribution disappears.

You can't tell which source backed which claim

- ▶ Ten sources merged into one smooth report
- ▶ Which source said this? Or was it fabricated?
- ▶ An unsourced claim is unverifiable



You can't tell a real finding from a fabricated one once everything is blended into smooth prose.

Claim-Source Mapping

Tie every claim to the source it came from.



Keep a claim → source link for each fact

- ▶ The final answer can show where each claim came from
- ▶ Like citations — claim stapled to its origin
- ▶ Makes the synthesis auditable



Structured attribution — claim + source kept together — echoes the structured hand-back from 5.3.

Conflict Annotation & Uncertainty

When sources disagree, say so — don't average.

Surface disagreement — and flag what's shaky

- ▶ Sources conflict? Show both, with their sources
- ▶ Don't silently pick one or average them
- ▶ Flag low-certainty claims instead of stating them flatly



Silently merging conflicting sources hides the disagreement. Surfacing it — and flagging shaky claims — is the honest move.

Temporal Framing

Old facts and new facts aren't the same fact.



Track WHEN information is from

- ▶ A 2019 figure and a 2026 figure aren't equally current
- ▶ Label the date / recency of each fact
- ▶ An old fact stated as current is quietly wrong



In a fast-moving topic, an old fact stated as current is a quiet error — temporal tags keep the answer honest.

Key Takeaways

Provenance & synthesis — and Domain 5 wrapped.

1

Keep a claim → source map

Merging sources loses attribution unless every fact points home.

2

Surface conflict & doubt

Show disagreements with their sources; flag uncertain claims.

3

Label recency

Temporal framing so old facts aren't stated as current.



End of Lecture 5.6 — Provenance & Multi-Source Synthesis